

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Mowbray
Faculty: Information Technology
Module Code: ITOPA3-33
Group: Individual
Lecturer's Name: Bongani Mahlangu
Student Full Name: Joshua Nehohwa
Student Number: EDUV4948467

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date

Lecturer's Comments:

--

Marks Awarded:	%

Signature	Date

ITOPA3-33 Operating Systems

Student	Joshua Nehohwa	Student number	EDUV4948467
Campus	Mowbray	Lecturer	Bongani Mahlangu
Module	ITOPA3-33	Assessment	Individual Assignment

Authorship and AI note: This is an AI-assisted study draft created from the supplied assignment brief and lecture slides. The student remains responsible for checking every claim, rewriting where required, and making a truthful declaration. The coversheet signature and plagiarism-report selection are intentionally left blank.

Question 1 - Operating-system system calls (20 marks)

1.1 Basic file-management system calls and their importance (8 marks)

When a customer requests a bank statement, the banking application should not communicate with the disk by itself. It should make controlled requests to the operating system through system calls. The first call is normally open. The application supplies the path or file identifier for the statement, and the operating system resolves that path, confirms that the process is allowed to access it, creates an open-file entry, and returns a file descriptor or handle. This is important in banking because the handle gives the application access to the authorised statement without exposing the whole storage device.

The application then uses read to request the statement data. The operating system coordinates the file system, cache, storage driver and physical disk, and copies the requested bytes into the application's memory. If the application needs a particular part of a large file, it can use seek to move to the correct offset. It may also use metadata calls to obtain the file size, dates or access information before displaying the statement. If the customer chooses to print or export the statement, a write operation sends output through the operating system's buffering and printer-spooling services. Finally, close releases the descriptor and any related kernel resources after the statement has been displayed or printed (Eduvos, 2026a; The Open Group, 2024).

These calls matter because they provide more than convenience. They enforce permissions, isolate one customer's process from another customer's data, coordinate simultaneous access, provide consistent error messages, support caching and buffering, and give the bank an auditable path for access. They also hide device-specific details, meaning the banking application can continue using the same interface if the bank changes its disks or printers. In short, the system calls turn a risky hardware operation into a controlled OS service.

1.2 Consequences of direct hard-drive and printer access (6 marks)

The developer's performance argument is understandable because avoiding a mode switch might appear to remove overhead. However, the small saving would introduce much larger risks. First, security would be weakened. Direct access could bypass normal permission checks, process isolation and protected file handles. A defect or malicious input could expose statements, transaction records or audit logs belonging to other customers. Modern operating systems deliberately separate user mode from kernel mode so that ordinary applications cannot freely control hardware (Microsoft, 2025a).

Second, data integrity and reliability would be threatened. Raw disk access can ignore file-system metadata, journaling, locks and cache-coherency rules. One incorrect block address could corrupt a statement, a transaction database or an entire volume. Direct printer control can also bypass the spooler, causing mixed jobs, lost pages or a blocked device. Third, concurrency becomes unsafe because thousands of banking sessions may issue conflicting operations without the OS scheduler, queues and locking mechanisms. Fourth, maintenance and portability become worse because the application would need device-specific driver logic for every disk controller and printer model. Finally, real performance may decline because the design gives up optimised OS features

such as caching, buffering, direct memory access, I/O scheduling and printer spooling (Eduvos, 2026a; Microsoft, 2025b).

1.3.1 CIO decision (2 marks)

I would reject the proposal. A banking system should not trade its security boundary, auditability and data integrity for a speculative performance improvement.

1.3.2 Justification (4 marks)

System calls are the controlled gateway between an application running in user mode and privileged operating-system services. The kernel validates a request, checks permissions, coordinates shared resources, invokes trusted drivers and returns a defined result or error. This design protects the application, the hardware and other processes at the same time. It is especially important in a regulated bank, where confidentiality, correctness, recovery and audit trails are more valuable than a minor reduction in call overhead. If performance is genuinely poor, the bank should profile the bottleneck and use supported options such as asynchronous I/O, batching, caching, faster storage or an approved driver update instead of bypassing the OS.

Question 2 - Multithreaded ShopFast server (25 marks)

2.1a Why the single-threaded server performed poorly (3 marks)

The server handled requests serially, so one request had to finish before the next request could make progress. During Black Friday, requests arrived faster than that one thread could complete them. The waiting queue therefore grew, response time increased, requests timed out and customers abandoned their carts. The design also failed to use the quad-core machine properly because only one thread could run application work at a time.

2.1b Blocking and its effect on clients (3 marks)

Blocking happens when a thread cannot continue until an event completes. For example, a ShopFast request may wait for a product database query, payment gateway response, disk read or slow network client. In the single-threaded design, that blocked request stops the only server thread. A customer browsing products must then wait even though their request is unrelated to the blocked payment request. This reduces both responsiveness and throughput (Eduvos, 2026b).

2.2a and 2.2b Three benefits linked to ShopFast (6 marks)

- Responsiveness and concurrency: if one worker waits for the payment gateway or database, other workers can still serve product searches and cart updates. This reduces timeouts and keeps the website responsive.
- Economy and resource sharing: threads share the same process code, configuration, connection pools and product cache. They are generally cheaper to create and switch than separate processes, so more of the server's resources remain available for customer work.
- Scalability across four cores: runnable request handlers can execute at the same time on different cores. This increases peak throughput when the pool size is controlled and shared resources, such as the database, do not become the next bottleneck (Eduvos, 2026b).

2.3a Task parallelism compared with data parallelism (3 marks)

Task parallelism assigns different tasks or functions to different processing units. The tasks may execute different code and work on different data. Data parallelism divides a dataset into parts and applies the same operation to each part at the same time. A useful example is calculating the same discount rule across four sections of a very large product list (Eduvos, 2026b).

2.3b Classification of the ShopFast server (3 marks)

The redesigned server mainly exhibits task parallelism. First, each HTTP request is an independent task belonging to a particular customer. Second, requests can follow different code paths: one thread may browse products, another may update a cart and another may process a payment. These tasks can run concurrently across the four cores even though their operations and data are different. Some repeated handlers may resemble data parallelism, but the overall server design is best classified as task parallelism.

2.4a Thread pool (2 marks)

A thread pool is a managed set of reusable worker threads. The server creates or maintains workers independently of individual requests. When a worker finishes one request, it returns to the pool and can process another request.

2.4b Why pooling is preferred (2 marks)

Creating a new thread for every request adds creation and destruction overhead and can allow a traffic spike to create thousands of threads. That consumes memory, increases context switching and can crash the server. A bounded pool reuses threads and limits the amount of work running at once, giving ShopFast predictable resource use. The best worker count must be measured: CPU-heavy work may stay near the core count, while I/O-heavy work may benefit from more than four workers (Oracle, 2025).

2.4c Role of the task queue (3 marks)

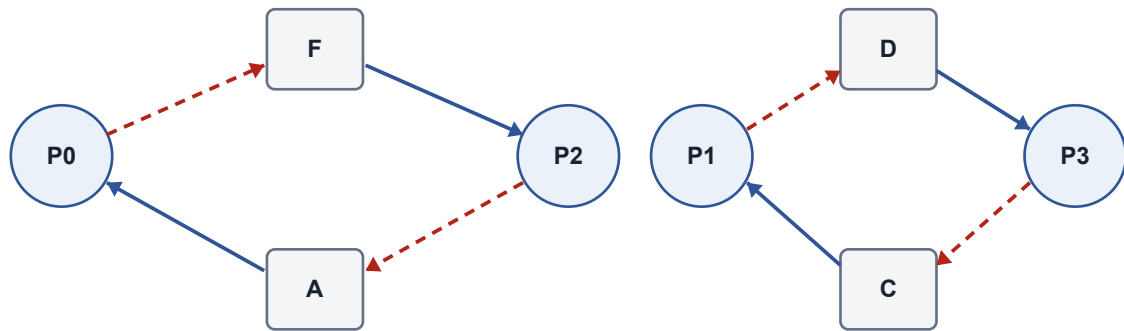
The task queue sits between request acceptance and worker execution. Each accepted HTTP request becomes a task and waits in the queue until a worker is available. The next worker removes a task, processes it and returns to the pool. The queue absorbs short bursts and can enforce first-in-first-out order or priorities. If it is bounded, it also provides backpressure: when both workers and queue are full, the server can reject or shed load in a controlled way instead of exhausting memory. ShopFast should monitor queue length and waiting time because a permanently growing queue means the system is overloaded, not healthy.

Question 3 - Deadlock in the manufacturing system (20 marks)

3.1 Resource Allocation Graph and deadlock result (10 marks)

Executing the resource requests line-by-line from P0 to P3 produces the following state. P0 obtains A, P1 obtains C, P2 obtains F and P3 obtains D. On the next line, P0 obtains B and P1 obtains one instance of E, while P2 waits for A and P3 waits for C. On the third request, P0 waits for F and P1 waits for D. The second instance of E is still free, but none of the blocked processes needs it next.

Resource Allocation Graph at the blocking point



Dashed red: request edge (process -> resource)

Blue: assignment edge (resource -> process)

Other held resources: B -> P0 and one instance of E -> P1. The second E instance is free.

The graph contains two cycles: P0 -> F -> P2 -> A -> P0 and P1 -> D -> P3 -> C -> P1. In the first cycle, P0 holds A and waits for F, while P2 holds F and waits for A. In the second cycle, P1 holds C and waits for D, while P3 holds D and waits for C. Since A, C, D and F each have one instance, the cycles confirm deadlock. P0 and P2 cannot release the resources needed by each other, and the same is true for P1 and P3 (Eduvos, 2026c).

3.2 Safe request order (5 marks)

I would impose one global resource order: A < B < C < D < E < F. Every process must request only in ascending order and should release in reverse order. The modified requests are:

Process	Safe request order	Reason for change
P0	A -> B -> F	Already follows the global order
P1	C -> D -> E	D is requested before E
P2	A -> F	A is requested before F
P3	C -> D	C is requested before D

This rule removes circular wait. A process holding a higher-ranked resource can never request a lower-ranked one, so a closed cycle of waiting edges cannot form. It also preserves the two instances of E; both are treated as the same resource class in the ordering.

3.3 Prevention or avoidance (5 marks)

Deadlock prevention is more suitable for this manufacturing system. The four processes repeat known operations and use a small, fixed set of resources, so the company can enforce a global request order in the control software. The rule is simple, deterministic and easy to test, which matters in a physical production line where a deadlock can stop machinery and delay output. Deadlock avoidance, such as the Banker's algorithm, would check whether each allocation keeps the system in a safe state. It can allow more flexible allocations, but it needs accurate maximum-demand information and adds a decision step to every request. That overhead and complexity are unnecessary here because the resource pattern is predictable. The prevention approach may reduce some concurrency, but the safety and operational simplicity justify that trade-off (Eduvos, 2026c).

Question 4 - File management and caching (35 marks)

4.1 Three advantages of directories (3 marks)

- Organisation: related files can be grouped by department or purpose, making employee records easier to locate and maintain.
- Name separation: different directories can contain files with the same name without a conflict because the full paths are different.
- Security and administration: permissions, backup rules and quotas can be applied to a directory and inherited by the files inside it. POSIX also describes files as a hierarchy in which directories form the non-terminal nodes (The Open Group, 2024).

4.2 Full path and Command Prompt line (4 marks)

The file is under Root (C:), then IT, then Software. Its full path is:

```
C:\IT\Software\Antivirus.exe
```

A Command Prompt line that runs it using the full path is:

```
"C:\IT\Software\Antivirus.exe"
```

The quotation marks are safe practice because they also work when a future directory name contains spaces. A fully qualified Windows path starts with the drive and follows each parent directory to the target file (Microsoft, 2025c).

4.3 HR and Public contents after the actions (4 marks)

HR directory	Public directory
Employees.xlsx Salaries.xlsx Contracts.docx Policies.pdf (copied in)	Policies.pdf Forms.docx

Public still contains Policies.pdf because the administrator copied the file rather than moving it. Deleting JanBackup.zip changes only IT\Backup. Renaming Expenses.xlsx changes only Finance, where the new name becomes Expenses_2026.xlsx. Therefore those two actions do not alter HR or Public.

4.4 Delete-on-logout compared with keep-until-deleted (8 marks)

A system that automatically deletes user files at logout or job termination has strong security and storage-management benefits. Temporary data does not remain for the next user on a shared computer, abandoned jobs do not fill the disk, and administrators spend less time cleaning old files. This approach suits kiosks, examination labs, temporary cloud jobs and systems that process sensitive short-lived data. Its main weakness is the risk of permanent data loss. A crash, forgotten save request or misunderstood policy can erase legitimate work. It also places a continuous burden on users to decide what must be preserved.

A system that keeps files until the user deletes them gives better continuity and recovery. Users can log out and continue later, records remain available for audits, and an accidental logout does not destroy work. This suits personal computers, office systems and business records. The disadvantages are that storage fills with stale files, private information remains exposed for longer, and users often fail to clean up or apply retention rules. Keeping everything can also increase backup cost and make searches less effective.

Neither extreme is ideal for every system. I would use a hybrid policy: automatically delete known temporary files and session caches; keep deliberately saved user documents; use quotas, retention periods and warnings; and provide a recycle or backup window for recovery. The policy should match the value, sensitivity and legal retention needs of the data.

4.5a Why systems treat file types differently (4 marks)

Some systems track file type because the operating system can then choose a suitable application, icon and permitted operation. For example, Windows file associations determine which application launches when a user opens a particular extension and which actions appear in the interface (Microsoft, 2021). File types can also help the OS distinguish regular files, directories, executables, devices and other special objects, improving validation and protection.

Other systems leave type information to the user or application, often through a filename extension or the file's internal format. This is simpler and flexible because the operating system can treat the content as a sequence of bytes. Systems that do not implement many file types reduce kernel complexity and avoid locking applications into a fixed type system. The trade-off is that applications must interpret the data correctly, and a misleading extension may cause the wrong program to be selected.

4.5b Which system is better? (4 marks)

There is no universally better design. Strong OS-managed typing is useful where safety, device control and a consistent user experience matter. Application-managed typing is better where flexibility, portability and simple file-system design matter. My preferred approach is a hybrid: the OS should keep a small set of essential structural types, such as directory, regular file, symbolic link and device, while applications use extensions, registered associations, MIME information or content signatures for detailed formats. This gives the OS enough information to protect the system without forcing every future document format into the kernel.

4.6a How caches improve performance (4 marks)

A cache keeps recently or frequently used data in a faster storage layer. When the requested item is already present, the system serves a cache hit instead of repeating a slower disk, network or main-memory access. This reduces average access time and traffic to the slower resource. Caches work because programs often show temporal locality, meaning recently used data is likely to be used again, and spatial locality, meaning nearby data is likely to be needed soon. File-system caches can therefore avoid repeated disk reads, while CPU caches reduce delays caused by main memory. Linux file-system caching also uses coherency and invalidation mechanisms so cached content does not silently replace correct file behaviour (The Open Group, 2024; Linux Kernel, 2026).

4.6b Why systems do not use unlimited or larger caches (4 marks)

Fast cache memory is limited, expensive and consumes chip area or system RAM. A larger cache can improve the hit rate, but the benefit eventually becomes smaller because not all workloads reuse the extra data. Larger structures may also take longer to search, use more power, require more metadata and make coherency between cores or devices more complex. At file-system level, a very large cache can take memory away from applications and must still manage dirty data, invalidation and recovery. It can also retain useless data and cause cache pollution. The best cache size is therefore workload-specific: Intel notes that cache effectiveness depends on whether the program's critical code and data fit the cache, and a poorly matched cache may provide little or even negative benefit (Intel, 2023).

Reference list

Eduvos. 2026a. ITOPA3-T11 Week 0 and 1 Lessons 1-4. Internal course slides.

Eduvos. 2026b. ITOPA3-T11 Week 2 Lessons 5-7. Internal course slides.

Eduvos. 2026c. ITOPA3-T11 Week 3 Lessons 8-10. Internal course slides.

Intel. 2023. Effective Use of Cache Memory. Nios II Processor Reference Guide. Available at:

<https://www.intel.com/content/www/us/en/docs/programmable/683836/current/effective-use-of-cache-memory.html> (Accessed 27 August 2026).

Linux Kernel. 2026. Network Filesystem Caching API. Available at: <https://docs.kernel.org/filesystems/caching/netfs-api.html> (Accessed 27 August 2026).

Microsoft. 2021. How File Associations Work. Available at: <https://learn.microsoft.com/en-us/windows/win32/shell/fa-how-work> (Accessed 27 August 2026).

Microsoft. 2025a. User Mode and Kernel Mode. Available at:

<https://learn.microsoft.com/en-us/windows-hardware/drivers/gettingstarted/user-mode-and-kernel-mode> (Accessed 27 August 2026).

Microsoft. 2025b. Windows Security Model for Driver Developers. Available at:

<https://learn.microsoft.com/en-us/windows-hardware/drivers/driversecurity/windows-security-model> (Accessed 27 August 2026).

Microsoft. 2025c. About Path Syntax. Available at:

https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_path_syntax (Accessed 27 August 2026).

Oracle. 2025. ThreadPoolExecutor, Java SE 24 API. Available at:

<https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html> (Accessed 27 August 2026).

The Open Group. 2024. POSIX.1-2024 General Concepts and System Interfaces. Available at:

<https://pubs.opengroup.org/onlinepubs/9799919799/> (Accessed 27 August 2026).

AI assistance disclosure

Tool: OpenAI Codex (GPT-5.6).

Use: The tool extracted and interpreted the assignment brief, reviewed the supplied Week 0-3 lecture slides, researched missing file-management details using official technical sources, drafted answers to Questions 1-4, produced the resource-allocation graph, filled the non-signature fields of the supplied coversheet, and formatted this PDF.

Student responsibility: The student must verify the content against the course material, revise the wording where required, complete any originality process, and make an honest decision about signing the coversheet. No signature or declaration response was inserted by the AI.

Prompt record

- "If you go to my downloads on my macbook, there is a file title ITOPA practical, its recent, extract it here please and go through it and let me know what it needs me to do."
- "Its due Friday night, so lets plan and get it done."
- "mowbray, eduv4948467, Bongani Mahlangu, use these files for now" with three Eduvos ITOPA slide decks.
- "continue till the end and answer all question in my tone. also that is the coversheet we should use" with the Individual Assessment Coversheet V1.2 PDF.